

Osnove distribuiranog programiranja — Pitanja za odbranu projekta

Primenjeno softversko inženjerstvo · 2026/2027. · Fakultet tehničkih nauka, Novi Sad

1. Način razmišljanja u distribuiranom sistemu

1. Objasnite razliku između poslovnog entiteta i procesa koji ga trenutno obrađuje u vašem sistemu.
2. Zašto lokalno stanje jednog procesa nije automatski izvor istine za ceo sistem?
3. Koje stanje u vašem projektu ima jasnog vlasnika i šta se dešava kad taj vlasnik privremeno nije dostupan?
4. Kako vaš sistem razlikuje ono što zna od onoga što samo pretpostavlja o stanju druge komponente?
5. Dajte primer gde je prekid mrežne veze tretiran kao očekivan scenario, a ne kao trajni kvar.
6. Koje stanje mora imati jedan jasan autoritet u vašem sistemu i šta se vidi kada taj autoritet privremeno nije dostupan?

2. Ugovori, simulatori i ponovljivost

1. Pokažite jedan ugovor (poruku/schema) u vašem sistemu i objasnite koje vrednosti dozvoljava.
2. Zašto slanje poruke nije isto što i potvrđen poslovni ishod?
3. Objasnite vaš simulator — koji tok i koje smetnje namerno proizvodi, i zašto je ponovljiv?
4. Kako primalac u vašem sistemu reaguje na poruku koju ne razume ili koja krši ugovor?
5. Zašto promena poruke bez verzije predstavlja rizik za postojeće korisnike ugovora?
6. Koja poruka u vašem sistemu najpre treba da postane stabilan ugovor i kakav simulator bi proverio njeno značenje?

3. Identitet, audit, observability i konfiguracija

1. Objasnite kako korelacioni identifikator prati jednu operaciju kroz API, red poruka i worker u vašem sistemu.
2. Pokažite jedan audit zapis — ko je izvršio radnju, kada i sa kojim ishodom?
3. Zašto audit nije zamena za svaki tehnički log, i obrnuto?
4. Kako vaš sistem proverava dozvolu zahteva — samo ulogu ili i pripadnost misiji/celini?
5. Koje konfiguracione vrednosti u vašem sistemu moraju biti vidljive timu i zašto?
6. Kako biste za jedan neuspešan zahtev spojili trag iz API-ja, reda poruka i workera?

4. Verzionisanje i failure-first testiranje

1. Objasnite razliku između kompatibilne i breaking promene ugovora na primeru iz vašeg projekta.
2. Pokažite jedan failure scenario (kašnjenje, duplikat, prekid) koji ste namerno testirali.
3. Zašto test treba da proveriti vidljiv poslovni ishod, a ne samo odsustvo izuzetka?
4. Da li ste neki bug reprodukovali pomoću FailureSimulator-a pre nego što ste ga ispravili?
5. Šta je Git tag `manual-core-baseline` u vašem projektu i zašto je taj trenutak važan?
6. Za koju radnju u vašem sistemu bi kasno pristigla poruka bila opasnija od izgubljene, i kako bi je sistem prepoznao?

5. Tok podataka i read modeli

1. Objasnite put jedne poruke od prijema, preko normalizacije, do usmeravanja u vašem sistemu.
2. Zašto nevalidna poruka mora imati vidljiv, a ne tih ishod?
3. Pokažite jedan read model i objasnite kako je prikazana njegova svežina (koliko može da kasni).
4. Kako vaš sistem sprečava dupli poslovni efekat kada ista poruka stigne više puta?
5. Zašto se vreme nastanka i vreme prijema poruke ne smeju mešati?
6. Koji podatak u vašem sistemu sme da kasni i kako bi korisnik trebalo da vidi da nije potpuno svež?

6. Komande, poslovi i idempotentnost

1. Objasnite stabilan identitet operacije u vašem sistemu — kako povezuje ponovljeni pokušaj sa istom namerom?
2. Pokažite dugotrajan posao (job) — kako se prati status kada worker padne i posao bude preuzet?
3. Zašto retry ne sme biti bezbedan za svaku operaciju bez granice ili identiteta zahteva?
4. Kako vaš sistem proverava da li kasna potvrda još odgovara aktuelnom stanju pre nego što ga izmeni?
5. Objasnite idempotentnost na konkretnoj komandi iz vašeg projekta.
6. Koja radnja u vašem sistemu bi imala neprihvatljivu posledicu ako se izvrši dva puta, i šta je njen stabilni identitet?

7. Rad bez veze i pouzdana isporuka

1. Objasnite kako vaš sistem čuva nameru korisnika kada nema veze sa udaljenim sistemom.
2. Pokažite outbox ili buffer mehanizam — koji je kapacitet i šta se dešava kada se popuni?
3. Kako deduplikacija na prijemu sprečava drugi poslovni efekat pri ponovnoj isporuci?
4. Objasnite replay mehanizam — da li prolazi kroz istu validaciju i audit kao nova poruka?
5. Šta je dead-letter tok u vašem sistemu i kada se poruka tamo završava?
6. Koju poruku biste obavezno sačuvali dok nema veze, a koju biste smeli da odbacite uz obaveštenje?

8. Koordinacija, protok i eventualna konzistentnost

1. Objasnite mehanizam (lease, epoch, fencing token) koji sprečava zakašnjelog vlasnika da piše nakon promene režima.
2. Zašto kratak prekid veze ne sme sam po sebi pokrenuti failover u vašem sistemu?
3. Pokažite gde vaš sistem koristi backpressure i šta se dešava kada se granica dostigne.
4. Objasnite repliku za čitanje u vašem sistemu — koliko sme da zaostaje i da li je to vidljivo korisniku?
5. Za koju odluku vaš sistem može koristiti zastareo prikaz, a za koju mora čekati autoritativno stanje?
6. Koji je najveći preostali (residual) rizik vezan za koordinaciju ili konzistentnost u vašem projektu?

9. Opšta i integrativna pitanja

1. Provedite nas kroz ceo lanac za vašu projektnu celinu: entitet i vlasništvo → ugovor → distribuirana operacija → failure/recovery scenario → test → evidencija.
2. Koja distribuirana odluka u projektu je bila najteža i zašto ste je doneli baš tako?
3. Koji trade-off (npr. dostupnost naspram konzistentnosti) ste svesno napravili i zašto?
4. Šta biste uradili drugačije da sada ponovo krećete sa distribuiranim delom projekta?
5. Objasnite zašto se realna oprema i mreža u projektu zamenjuju kontrolisanim, ponovljivim simulatorima.